ACTIVITY 3

System Development and Repository Deployment and Progress Documentation

From Database Design to a Working System

Course	COMP 018 – Database Administration	**Activity Type**	Group Activity
Database Platform	PostgreSQL	**Core Validation**	Cross-group local setup and testing; PDF progress documentation

Activity Overview

Students will transform their previously proposed system designs into working applications and publish each project in the group's own Git repository. Every group will register its repository in a shared Google Sheets tracker so that all projects can be found, monitored, and tested by other groups. Each project must include the necessary database setup scripts, migrations, initial sample data, configuration instructions, and database-related documentation. Each group must also submit a PDF progress document that records how the system was built, together with the link to its project repository.

The project must be tested by the other groups to verify that it can be installed and executed on a different local environment without relying on the original developers' machines.

I. Learning Objectives

1. Implement a working application based on the approved system design and database schema.
2. Prepare reproducible database setup procedures using SQL scripts, migrations, and seed data.
3. Document database configuration, application connectivity, and required installation steps.
4. Document the user roles inside the application and map them to the intended database roles used in Activity 4.
5. Use Git to manage source code, host the project in the group's own repository, and share it by registering it in the class Google Sheets tracker.
6. Deploy and test the other groups' projects locally using only each repository and its documentation listed in the tracker.
7. Document development progress, decisions, and evidence in a clear PDF report that accompanies the project repository.
8. Evaluate AI-generated code and database scripts for correctness, security, and consistency with the approved design.

II. Activity Instructions

Part A – System Implementation

Use your previously approved system proposal from Activity 2 as the basis for development.

1. Review your system requirements, ERD, relational schema, and business rules.
2. Identify the user roles of the system (for example administrator, staff, or customer) and determine which functions and data each role needs.
3. Implement the application using your chosen programming language or framework.
4. Implement the database using PostgreSQL, ensuring that tables, relationships, constraints, and data types match the approved design.
5. Implement the essential functions of your system, including appropriate data creation, retrieval, updating, and deletion operations.
6. Test the application to verify that its database operations work correctly.

AI tools may be used to generate code, suggest database structures, troubleshoot errors, and improve documentation. However, each group is responsible for validating the generated output and understanding the code and database scripts submitted.

Part B – Database Setup and Documentation

Your group's repository must contain all resources needed to initialize the database and run the application on another computer.

Required file/documentation	Description
README.md	Project overview, prerequisites, installation steps, configuration, and execution instructions.
.env.example	Example environment variables required by the application, without actual credentials.
Database migrations or SQL scripts	Files and instructions for creating the database schema and its objects.
Seeders or sample SQL data	Initial records that allow the system to be tested immediately.
Database documentation	Database name, schemas, tables, relationships, keys, constraints, and relevant database objects.
Application user roles documentation	The user roles inside the application, what each role can do, and how each maps to the intended database roles (see below).
User and role setup documentation	Intended database group roles, login accounts, and privileges for each application role, without committing real passwords.
Troubleshooting guide	Common setup errors and corresponding solutions.

Organize resources according to your chosen framework. For example, Laravel projects can use migrations and seeders, while other projects may use SQL initialization scripts or their framework's equivalent.

Application user roles documentation

Document the roles that people play inside the application in the README or a docs/user-roles.md file. Separate the roles inside the application (what a person can do after logging in) from the roles inside PostgreSQL (what a database account can do). This documentation is the starting point for the database users, group roles, and privileges created in Activity 4, so keep the names and responsibilities consistent.

Application role	Responsibilities in the application	Functions and data access	Intended database role (example)
Administrator	Manages users and system settings.	Create, read, update, and delete on user management; read on reports.	Elevated group role, not a superuser.
Staff / data entry	Encodes and updates records.	Create and read on records; update limited to assigned data.	app_runtime_grp

Application role	Responsibilities in the application	Functions and data access	Intended database role (example)
Viewer / auditor	Views records and reports.	Read-only on approved tables and views.	app_readonly_grp
Application runtime (system account)	The account the application itself uses to connect to the database.	Only the operations the application code needs.	app_runtime_grp

The roles above are examples; replace them with your own system's roles. An application role is not the same as a database account: not every end user needs a PostgreSQL login, and the application usually connects through a runtime account. Do not commit real passwords.

> **Security reminder:** Do not commit .env files containing real credentials, database passwords, API keys, or other secrets. The .env.example file must contain placeholders only.

Part C – Project Repository Tracker and Progress Documentation

Instead of publishing to a single centralized repository, each group keeps and publishes its project in its own repository (for example, a GitHub, GitLab, or similar Git hosting account of the group). Groups share their repositories through one Google Sheets tracker maintained for the class. The tracker is the single place where every group's repository link, status, and testing results can be found.

Step 1 – Prepare your own project repository

1. Create one repository for the group and add every member as a collaborator.
2. Include the complete application source code and necessary database resources.
3. Use meaningful commits that reflect development progress; every member should contribute commits.
4. Include the required documentation and sample data listed in Part B.
5. Exclude unnecessary files such as dependencies, generated build files, local database dumps containing sensitive information, and secret configuration files.
6. Make the repository accessible to all other groups (public, or private with every other group's members invited). Verify access using an account that is not part of your group.

Step 2 – Register in the Google Sheets tracker

The instructor shares one Google Sheets tracker with the whole class. Each group adds and maintains one row with the information below, and updates it whenever its repository or status changes.

1. Add your group's row to the tracker before the registration deadline set by the instructor.
2. Paste the repository URL and confirm that the link opens for someone outside your group.
3. Update the status and the progress documentation link as work advances; do not delete or edit other groups' rows.
4. Respond to testing feedback recorded by the testing groups and note the resolution in the tracker.

Tracker column	What to enter
Group no. and name	Group number, project/system name, and member names.
Repository URL	Link to the group's own repository, with the access type (public or private).
Branch/tag to test	The branch or release tag the testing groups should clone.
Tech stack	Language/framework and PostgreSQL version used.
Status	Not started, In progress, Ready for testing, Tested, or Revised.
Progress document	Link to the group's PDF progress documentation.

Tracker column	What to enter
Testing log	Pointer to the Activity 3 Testing Log tab, where the other groups record their results for this project.
Verification result	Latest overall result (Pass, Pass with issues, or Fail) after the owning group has addressed the testing feedback.
Last updated	Date of the most recent update by the owning group.

The tracker also contains an Activity 3 Testing Log tab, used in Part D. Each testing group adds one row per project it tests and edits only its own rows.

Step 3 – Submit the PDF progress documentation

Each group must produce one PDF document that records all of its progress on the project and submit it together with the link to its own project repository. The PDF must include the following sections.

Section	Contents
Cover and team	Project title, group number, members with roles, and repository and tracker links.
System overview	Purpose, scope, and main functions, tied back to the approved Activity 2 proposal.
User roles	Application user roles, their functions and data access, and their mapping to the intended database roles and accounts.
Development progress log	Dated milestones or weekly entries showing what was built, who worked on it, and key commits.
Database implementation	ERD, schema summary, migrations or SQL scripts used, seed data, and any changes from the approved design with reasons.
Application evidence	Screenshots of the working system and results of the create, retrieve, update, and delete tests.
Setup and configuration	Condensed installation steps, environment variables (placeholders only), and database user and role setup.
Issues and resolutions	Problems met during development and setup, and how they were solved.
Cross-group verification	Summary of the testing performed on the other groups' projects, the feedback received on your own, and the fixes made.
AI usage declaration	AI tools used, tasks performed with them, and how the group validated or corrected the output.

The PDF must not contain real passwords, API keys, or other secrets. Screenshots should be checked before submission to confirm that no credentials are visible.

> Recommended arrangement: each group maintains its own repository, and the Google Sheets tracker serves as the class directory. This keeps project history and permissions with the group while still making every project easy to find and test. Set the sheet so that groups can edit only their own rows.

Part D – Cross-Group Deployment and Verification

Your group will test the projects of all the other groups. For each project, use a separate local environment and only the repository and documentation listed in the Google Sheets tracker. If a project cannot be installed, document the blocking problem and continue with the remaining projects.

1. Open the tracker and clone the repository of every other group (not your own) from the listed URLs.
2. For each project, review its README, PDF progress documentation, and user roles documentation, and identify the required software, dependencies, and configuration.

3. Configure each application and its own PostgreSQL database locally, keeping the databases separate.
4. Execute the database migrations or SQL initialization scripts.
5. Load the sample data.
6. Start each application, test its essential functions, and check that the documented user roles can be exercised (for example, by logging in with sample accounts).
7. Record missing instructions, configuration errors, database inconsistencies, or other issues encountered for each project in the Activity 3 Testing Log tab of the tracker.
8. Provide feedback to each project's original developers and document whether each project can be successfully reproduced. Record the result for each project in the Activity 3 Testing Log.

Record every test in the Activity 3 Testing Log tab of the tracker, using one row per project tested.

Testing Log column	What to enter
Project tested	Group number and project name.
Testing group	Your group number.
Repository cloned	Repository URL and the branch or tag used.
Setup result	Success, Success with issues, or Failed, with a short note.
Application roles checked	Which documented user roles were exercised, and the result.
Issues found	Missing instructions, configuration errors, or database inconsistencies.
Feedback sent	Date feedback was sent to the owning group.
Last updated	Date of your most recent update.

> The testing group must not silently modify the original group's repository to make the project work. Required corrections should be documented and communicated to the project owners.

III. Required Deliverables

- **Working application —** A functioning implementation of the approved system.
- **Group project repository —** The group's own repository with complete source code, database resources, and required documentation, registered in the Google Sheets tracker.
- **Progress documentation (PDF) —** A PDF recording all of the group's progress, submitted together with the repository link.
- **Database documentation —** Schema, migrations or SQL scripts, sample data, configuration, application user roles, and intended database user/role setup.
- **Setup verification report —** For each tested project, evidence that it was successfully installed and run, or a detailed report of problems preventing deployment.
- **AI usage declaration —** AI tools used, tasks performed with them, and validation or corrections made by the group.

The cross-group verification report should be completed by each group that tests another group's project, with the original group given an opportunity to address the findings.

IV. Assessment Rubric

Criteria	Points
System implementation and functional correctness	25
Database implementation and consistency with the approved design	20
Database setup, migrations, sample data, user roles, and documentation	15

Criteria	Points
Group repository, tracker registration, and version control practices	10
Progress documentation (PDF): completeness, clarity, and evidence	10
Successful cross-group deployment and verification	10
AI usage transparency, validation, and technical understanding	10
TOTAL	**100**

> A project should not receive full credit for documentation merely because the files exist. The setup instructions must work when followed by another group, and the PDF progress documentation must reflect actual work shown in the repository history.

V. Submission Checklist

- ☐ The application runs and its essential functions work.
- ☐ The database matches the approved design.
- ☐ Database migrations or initialization scripts are included.
- ☐ Sample data can be loaded successfully.
- ☐ README contains complete setup and execution instructions.
- ☐ Environment variables use placeholders and no real secrets are committed.
- ☐ Database users and intended privileges are documented.
- ☐ Application user roles are documented and mapped to the intended database roles.
- ☐ The project is published in the group's own repository, and the testing group can access it.
- ☐ The repository URL, status, and progress document link are entered in the Google Sheets tracker.
- ☐ The PDF progress documentation is complete and contains no secrets.
- ☐ All other groups' projects have been deployed locally and documented in the testing log.
- ☐ AI assistance and validation activities are declared.

Final Note

Keep Activity 3 focused on development, database reproducibility, documentation, and repository collaboration. Comprehensive security implementation and backup/recovery procedures will be addressed in later DBA activities. The key success criterion is that another group can find the project in the tracker, clone it, initialize its database, load the sample data, and run the application without requiring the original developers to configure their machines.